

section 3.7: Break and Continue

pages 64-65

Note that a `break` inside a `switch` inside a loop causes a break out of the `switch`, while a `break` inside a loop inside a `switch` causes a break out of the loop.

Neither `break` nor `continue` has any effect on a brace-enclosed block of statements following an `if`. `break` causes a break out of the innermost `switch` or loop, and `continue` forces the next iteration of the innermost loop.

There is no way of forcing a `break` or `continue` to act on an outer loop.

Another example of where `continue` is useful is when processing data files. It's often useful to allow comments in data files; one convention is that a line beginning with a `#` character is a comment, and should be ignored by any program reading the file. This can be coded with something like

```
while(getline(line, MAXLINE) > 0) {
    if(line[0] == '#')
        continue;

    /* process data file line */
}
```

The alternative, without a `continue`, would be

```
while(getline(line, MAXLINE) > 0) {
    if(line[0] != '#') {
        /* process data file line */
    }
}
```

but now the processing of normal data file lines has been made subordinate to comment lines. (Also, as the authors note, it pushes most of the body of the loop over by another tab stop.) Since comments are exceptional, it's nice to test for them, get them out of the way, and go on about our business, which the code using `continue` nicely expresses.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)